ProactiCode: A Quality-Aware and Technical Debt Prediction and Prevention AI Code Assistant

Saanvi Anup K, Saritha S

Abstract

Coding AI assistants can help developers generate code more quickly and can easily create a software. However, this vastly generated code may introduce quality problems in the software, including code duplication, increased complexity, code duplication, and possible technical debt. The research paper on Cursor AI has reported a connection between AI-assisted development and deterioration in code quality .These findings motivate us in the development of a quality aware coding AI assistant that evaluates code quality alongside productivity of development.

We propose **ProactiCode** which is a Quality Aware AI Code Assistant designed to detect and reduce any potential technical debt in AI assisted coding programs or software development. The new framework will measure the code complexity, static code analysis, automated line by line testing, risk assessment and default error management. A rule-based risk scoring system for the code will be established to set a transparent baseline. Also, machine-learning models will be explored to predict potential quality risks in code when enough labelled data are available.

This AI assistant will classify changes in code into risk categories and show recommendations . The proposed model will be evaluated against a static analysis only baseline using code quality metrics, prediction performance, test results, and processing time. The goal is to check if integrating assessment of risk and recommendations improvement in recommendations can help software developers maintain code quality while benefiting from AI assisted programming.

1. Introduction

There is a rise in use of AI coding assistants to generate code, recover errors, suggest implementations, and fastens software development. But faster code production does not necessarily guarantee reliable or securable software.

Based on the Recent research paper, Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects, shows how the Cursor AI's adoption in GitHub affects the development of software .The study reports an increase in number of lines of code added and identifies increases in uncovered code-quality and cognitive complexity. It also shows how the development entirely is affected when there are code quality related issues. These research studies shows the importance of checking and

assessing code quality alongside productivity when coding assistants are used for programming.

Many office workflows rely on software developers or conventional AI tools to identify problems after code has been generated. Even though these tools can detect specific issues, they don't provide a proper assessment of the risk with a change, explain its quality enhancements, recommend a required improvement, and verify the modifications on the result.

To address this challenge, we propose **ProactiCode**, a quality aware AI assistant framework that combines step by step code analysis, technical risk assessment, simultaneous code testing, and providing recommendations. Other than emphasizing exclusively on how much fast code an AI assistant produces, our proposed system will examine whether the generated or modified code meets defined quality requirements of the program.

Research Objectives

The primary objectives of the proposed work are:

- To understand whether ML models can predict quality risks from code and change the related features.
- To develop a technical debt risk scoring using measurable code-quality indicators simultaneously.
- To check code quality problems in AI generated/modified code.
- To provide explanation for code recommendations for enhancing code and code improvement.
- To implement automated tests to check if the implemented modifications perform expected outcomes.
- To compare the our proposed AI assistant framework with previous frameworks like Cursor AI's framework.

2.Related Work

2.1 AI-Assisted Software Development and Code Quality

AI coding assistants have changed how software is made and how developers produce codes and programs. By generating code from English language instructions and existing code snippets, these AI systems can reduce manual code implementation effort. However, generated code still needs to be evaluated to check if it is correct, readable, maintainable and if its secure.

The research paper motivating this project**, Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects**,

examines program outcomes achieved with Cursor AI adoption in programs in GitHub. The analysis on this shows a potential disruption between faster code production and software aaquality measures.

The study motivates further work on practical mechanisms that can detect quality risks during development. However, measuring the relationship between AI adoption and software quality is different from building an interactive system that evaluates individual code changes and recommends verified improvements. **ProactiCode** aims to investigate this implementation-oriented direction.

2.2 Static Code Analysis and Software Metrics

Static code analysis looks at source code without running the program. Tools like Ruff, Pylint and SonarQube can find coding problems, unusual patterns and issues with how easy the code's to maintain. Other software metrics can measure things like complexity, duplication of code and how long each function is.

Individual alerts or complexity numbers do not always mean the program has a problem. Understanding them depends on the programming language the project executes on, the code situation and what testing is needed.

The suggested framework will use indicators instead of just one. It will also tell the difference, between problems found and the risk they might cause. This way it will not assume that every complicated function or static code issue is a problem.

2.3 Technical Debt and Machine-Learning-Based Risk Prediction

Technical debt tells the work and the cost that comes after the project is released because of design or coding choices made early on. These choices make the software harder to change or maintain. Sometimes it happens if the development was too fast and testing was not good or the design wasn’t apt.

We can use ML to try and predict problems in software like any bugs or quality issues by looking at development data. Methods like Logistic Regression, Random Frest and gradient boosting classifiers can find patterns between features and known outcomes.

A model can't learn to predict future defects well just from data that only includes complexity numbers and static-analysis warnings. It needs labels like confirmed bugs or fix-related changes that show what happens in the code. So, the proposed work will first build a rule-based system to score risk.

2.4 Research Gap

Existing analysis tools can catch code problems and defect-prediction methods can understand risks based on data in code line. These don’t usually work together in one smooth interactive workflow. That workflow should check code changes explain why they’re risky suggest fixes and then test those fixes to make sure they help. The Cursor AI study mostly

looks at how AI affects GitHub repositories project but It doesn't build a system to prevent code quality issues before they happen or when the project is published.

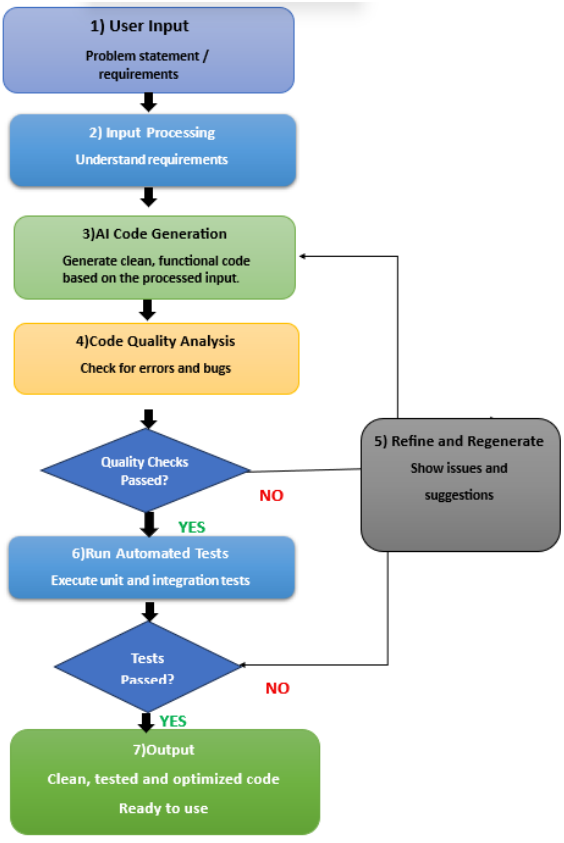
ProactiCode wants to fix this by:

- Integrated risk assessment: Right now, different code quality signals are separate. We need to bring them into one clear view of how risky a code change really is.
- Guidance on each action: When problems are found developers need plain explanations and specific steps to fix them. Not just warnings, directions are required to fix code.
- Validated recommendations: After suggesting a fix we should automatically run tests. We also need to compare code quality before. After the fix to see if things improved.
- Comparative framework: The new framework must be tested against a simpler version. That way we can tell if the extra features help or not.

These gaps push us to build a testable system. We aren't assuming that adding machine learning will automatically make software better. We want proof.

3. Proposed Work

We propose to develop and evaluate **ProactiCode** as a modular framework for analysing source code, assessing potential quality risks, recommending improvements, and validating modifications. The initial implementation will focus on Python repositories to keep the project manageable and enable reproducible experiments.



3.1 Code Collection and Quality Analysis

In the first phase the input dataset and the code-analysis pipeline will be set up.

- Dataset and inputs: The system can take Python source files or changes to a repository, along with the relevant test files and, if available, historical commit information. Public repositories may be used if they comply with their licences and data-collection restrictions.
- Using static analysis, tools such as Ruff or Pylint will detect coding errors and suspicious patterns; Radon will be used to compute the cyclomatic complexity and other related code metrics, and where possible, indications of code duplication and function size will also be included.
- Testing: Pytest will run the available unit tests to check whether the current code passes the existing test suite; test coverage can be measured using a coverage tool when it is appropriate.
- Security checks: Bandit can be integrated to detect certain security-related patterns in Python code.
- Feature extraction: The system will, for each file or change that is analysed, extract indicators such as the number of warnings, complexity, duplication, function length, the size of the change, the test results, and the available coverage information.

The resulting feature set will provide a consistent representation of code quality for subsequent risk assessment.

3.2 Technical-Debt Risk Assessment

The second phase will develop a risk-assessment mechanism that combines the extracted indicators into an understandable assessment.

Initial approach: A rule-based scoring system will establish the baseline. Each indicator will contribute according to defined thresholds and weights. For example, a change introducing several high-severity warnings, a large increase in complexity, and failing tests may receive a higher risk score.

A conceptual score can be expressed as:

$$R = \sum_{i=1}^{n} w_i x_i$$

Where:

R: the risk score,

N: total number of quality indicator

W_i: weight of i-th indicator

X_i : normalised i-th quality indicator.

The results will be divided into categories such as Low Risk, Moderate Risk, and High Risk. These categories will indicate the estimated quality issues, not assure that the code is correct or defective. If there is enough historically recorded labelled data available, supervised models such as Logistic Regression, Random Forest, and gradient boosting will be used to estimate the probability of a specific future quality result, the desired outcome and the period for the prediction being clearly specified prior to the training process. The rule-based baseline will be used to compare the machine-learning models to see whether they provide measurable improvements in predictive ability.

3.3 Explainable Recommendations and Refactoring Support

In the third phase, the results of the analysis will be turned into specific recommendations. For every issue that has been flagged, ProactiCode will show the file or function in question, the appropriate quality indicator, the reason for the warning, and a suggestion for an improvement.

Examples of such improvements may be to simplify nested conditional logic, reduce code that is repeated, split up a function that is too large, or deal with a high-severity static-analysis warning. The recommendations will at first be based on rule-based templates associated with the issues detected, and a optional AI-based component could later produce more contextual suggestions.

The system will not treat a suggested modification as correct by default; when refactoring is carried out, the original version will be kept, the changes will be made in a separate working copy or branch, and the resulting code will follow the same analysis and testing pipeline. The system will be able to compare the original and altered versions by means of measurable indicators instead of depending solely on the way the suggested code looks.

3.4 Automated Testing and Quality Gates

In the fourth phase a validation mechanism will be introduced to determine if code changes satisfy the established quality requirements. Where possible, the system will carry out the available unit tests both before and after a proposed modification and will also rerun the relevant static-analysis and complexity checks, comparing the resulting metrics.

Based on configurable criteria, the system will provide one of three recommendations:

- Except for review: No blocking condition has been configured, and the change satisfies the chosen checks.
- Flag for review: This change has moderate risk indicators or unresolved warnings.

- Requests for changes: If there is a blocking condition, for example, if the required tests have failed or a critical configured finding exists, action is needed.

These results will be used to assist with the review rather than take the place of the developer's judgement. A low-risk score will not cancel out a failed required test or a critical security issue. The quality-gate mechanism will keep a record of whether a proposed modification passes the checks that have been configured and of whether the change causes the selected metrics to improve, preserve, or worsen.

3.5 Quality Monitoring Dashboard

The fifth phase will develop a dashboard that presents analysis results in a consolidated interface.

The dashboard will include:

- Overall risk category and risk score.
- Static-analysis warnings grouped by severity.
- Complexity and duplication indicators.
- Test execution status and coverage, where available.
- Explanations and recommendations for flagged issues.
- Before-and-after comparisons for modified code.
- Historical quality trends across analysed revisions.

The interface will help developers understand which issues require immediate attention and whether quality indicators improve after a modification.

A lightweight implementation using Python and Streamlit will be considered suitable for the initial prototype. The analysis engine will remain modular so that the interface can be replaced or extended later.

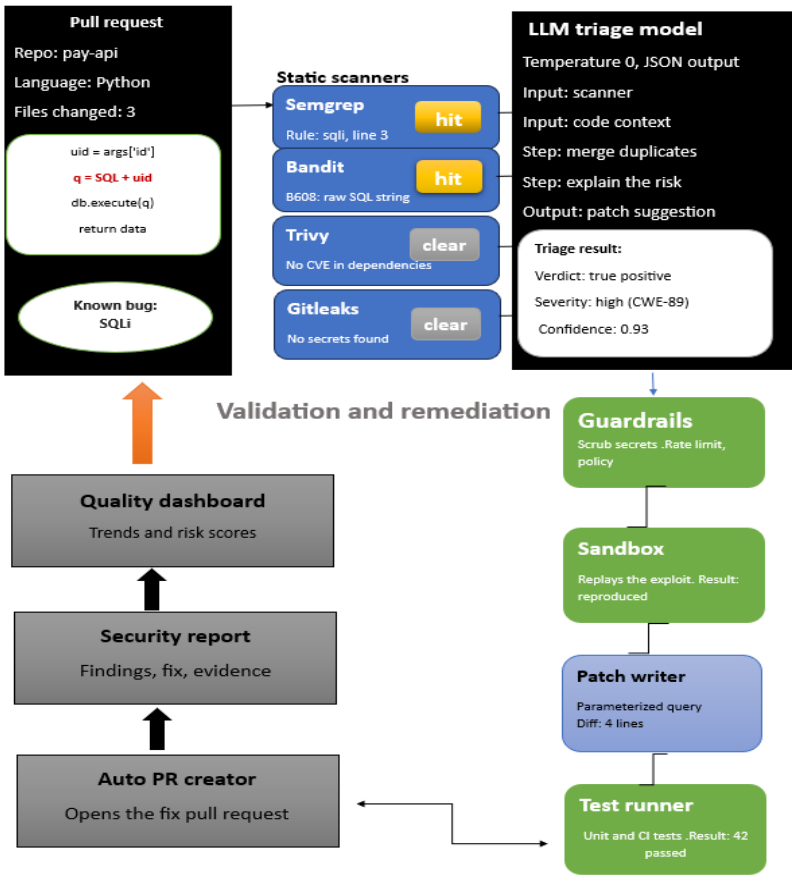
3.6 Experimental Design and Baselines

To assess whether ProactiCode provides practical benefits, the system in question will be compared with a series of baselines that are increasingly improved.

In the case of Baseline A, which involves static analysis alone the source code is analysed by means of static-analysis and metric tools that have been configured, and the results are given out without carrying out an integrated risk assessment or providing automated recommendations.

The indicators are combined using a rule-based system to produce a risk score and risk category. The proposed system **ProactiCod**e incorporates code analysis, risk assessment, explainable recommendations, automated test validation, and comparisons before and after an action.

If suitable labelled data are available, the supervised machine-learning component will be assessed separately. Identical evaluation samples and similar execution conditions will be applied in the various baselines. This arrangement will allow the contribution of the extra components to be isolated.



4. Evaluation and Limitations

4.1 Evaluation Metrics

The system that has been proposed will be assessed by means of several different dimensions.

- We will measure the changes in the code-quality indicators, such as the number of static-analysis warnings, complexity, and duplication, both before and after the recommendations have been applied.
- The results of the testing will record the pass rate of the tests, any new test failures, and cases of regression. Coverage will be reported once reliable coverage data are available
- In terms of prediction performance, precision, recall, the F1-score and the PR-AUC will be used when labelled data are available for supervised learning. Calibration measures will also be given when probability outputs are produced by the model.
- The effectiveness of the recommendations will be assessed by looking at the proportion of the suggested changes which are accepted following review, the

proportion that result in an improvement to the specified metric, and how often regressions occur.

- In terms of efficiency, the time taken for the analysis, the time required for validation, and the effort involved in the developer's review will all be recorded so that they can be measured consistently.

The results will be presented using the relevant summary statistics and, where possible, multiple runs together with paired comparisons will be made in order to determine whether the differences observed are consistent or merely due to a specific sample.

4.2 Experimental Procedure

The evaluation will follow these steps:

1. Choose the Python repositories or code changes that are eligible and record the criteria used for inclusion.
2. Carry out the static-analysis baseline and note the original metrics.
3. Use the rule-based risk assessment on the same examples. For the issues that have been selected, make recommendations and, where possible, put them into effect in a separate branch or copy.
4. Carry out a further static analysis and automated tests on the altered code.
5. Look at the quality indicators, the results of the tests, the processing time, and the effectiveness of the recommendations for each of the baselines.
6. If you have enough labelled data, train and test machine-learning risk predictors using repository-aware or chronological data splits. This helps reduce data leakage.
7. Include cases in which the system fails, the various trade-offs, and all instances where the proposed system does not provide an improvement over the baseline.

A recommendation cannot be regarded as successful just because it lowers the complexity score; it also must maintain the expected functionality and pass the relevant validation checks.

4.3 Expected Contributions

The proposed work is expected to deliver:

- A pipeline for the analysis of code quality in Python projects that is modular.
- A baseline that is transparent and is based on measurable quality indicators. An optional machine learning part predicts quality-related results when you have the right labels

- A workflow for explaining the recommendations and validating the refactoring.
- A dashboard used for examining code-quality risks and changes over time.
- A comparison based on empirical evidence between approaches that use static analysis alone and those that are integrated and quality aware.

These contributions are provisional and will be verified by means of implementation and evaluation; no assertions regarding improvements in software quality, prediction performance, or development efficiency will be made unless they are supported by experimental results.

4.4 Limitations and Future Scope

There are many limitations with the proposed system. Firstly, the warnings obtained from static analysis and the complexity measures are inadequate stand-ins for both maintainability and correctness. A high complexity score does not necessarily mean that there is a defect, and a low score does not ensure that the code is reliable.

Secondly, supervised risk prediction relies on the availability and quality of historical labels; if the data gathered do not include meaningful future outcomes regarding defects or corrective changes, the initial implementation will still be a rule-based risk assessment system rather than one that claims to predict actual defects.

Thirdly, the fact that the initial focus is on Python tends to limit the ability to generalise to other programming languages and development environments. Moreover, the results could be influenced by the repositories chosen, the testing quality, the tool configurations, and the project conventions.

Fourthly, automated refactoring may inadvertently alter the behaviour of a program. Although the suggested approach of making isolated changes and using automated tests helps to reduce this risk, it cannot ensure that no regression will occur, particularly in the case of incomplete tests.

In the end, the initial prototype will be concerned with assessing and preventing code problems rather than serving as a replacement for an existing AI coding assistant. Possible areas for future work include integrating with development environments, providing support for additional languages, improving defect prediction, and testing on larger datasets.

References

[1] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu

Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects.

[2] Maja Vukovic, Rangeet Pan, Tin Kam Ho, Rahul Krishna

Usage, Effects and Requirements for AI Coding Assistants in the Enterprise: An Empirical Study

[3] Annie Vella, Kelly Blincoe

The Impact of AI Coding Assistants on Software Engineering: A Longitudinal Study

[4] Lehui Li, Ruixuan Jia, Guo-Ye Yang, Jia Li

An Empirical Study of Proactive Coding Assistants in Real-World Software Development

[5] IBM Technology YouTube video;

Spec-Driven Development: AI Assisted Coding Explained